

Reinforcement Learning for HVAC Optimization and Energy Management

Isuri Amalsha Ranawaka, Nowshin Binte Jamal, Sambhav Siwakoti, Santosh Luitel

Department of Computer Science, University of South Dakota, Vermillion 57069, USA

{isuriamalsharanawaka, nowshinbinte.jamal, sambhav.siwakoti, santosh.luitel}@coyotes.usd.edu

Abstract

Reinforcement learning offers a great way to optimize residential HVAC systems, which account for a considerable share of household energy or costs. This research examines the application of RL to HVAC control and assumes that using RL would not only allow for reduced energy consumption but also maintain the cost to satisfy indoor temperature stability. The research process includes developing a unique RL environment for HVAC control and implementing and comparing the performance of a set of deep reinforcement learning algorithms: DQN, DDQN, PPO, and SAC. To assess the performance of the algorithms, the analysis utilized a time series data split, as well as hyperparameter tuning methods, including Grid Search. The evaluation of the experimental performance results indicated that while the Grid Search variant of PPO exhibited the highest energy and cost savings, the SAC demonstrated superior temperature deviation, which would suggest a tradeoff amongst algorithms. All the optimized models in the experiments appeared to consistently outperform all the unoptimized models. Overall, the research concludes that RL shows great promise for providing intelligent, energy-efficient programming and control of HVAC systems, and future research will continue to advance control methods and apply them to easily integrated real-world control solutions.

Code — <https://github.com/amalsha27/RL-Project>

Datasets —

<https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction>

Introduction

Energy consumption for domestic use is a significant global issue, and Heating, Ventilation, and Air Conditioning (HVAC) systems are one of the biggest consumers of energy in residential buildings, often accounting for around 50% of the energy usage of the household. Traditional HVAC control systems operating inefficiently through devices that turn on unnecessarily, and how they're controlled without the ability to adapt to changing weather and occupancy both contribute to the considerable amount of energy wasted, and thus cost consumers money and have further environmental consequences. This gap needs addressing through intelligent and adaptive control systems to promote energy efficiency and sustainability domestically.

This research examined the issue of inefficient HVAC control in homes, leading to unnecessary waste of energy and occupant discomfort. The primary objective was to use reinforcement learning (RL) to automate HVAC system

control to reduce energy consumption and costs while maintaining indoor temperatures within acceptable limits, while creating as little deviation as possible. To achieve this goal, the study presented the following research questions: How can reinforcement learning be effectively applied to HVAC optimization? And can energy consumption and cost be reduced with reinforcement learning added to HVAC control?

This work has three main contributions. First, it offers a unique reinforcement learning environment for HVAC control, allowing RL agents to be trained and assessed in a smart home simulation. Second, it includes an exhaustive comparison of four reinforcement learning algorithms, DQN, DDQN, PPO, and SAC, in this environment. Finally, it offers insights into the applicability of different RL agents, as well as the influence of tuning hyperparameters, in managing energy efficiency and comfort in HVAC systems for residences.

The rest of the paper is structured as follows: Section 2 provides essential Background information on reinforcement learning and HVAC systems. Section 3 provides an overview of Related Work relevant to the application of RL for energy management and HVAC control. Section 4 describes the Preliminaries and Mathematical Framework, detailing the problem formulation including state space, action space, and reward function. Section 5 outlines the Methodology, including algorithms used and experimental setup. Section 6 contains a detailed description of the Experiments. Section 7 contains the Reports and Results of the experiments. Section 8 describes the Discussion of the findings, challenges, and limitations. Section 9 refers to their Future Work. Section 10 contains the Conclusion of the research and a list of References.

Background

This section offers fundamental knowledge in reinforcement learning and is specific to the environment and components in relation to the HVAC optimization problem this research addresses.

Reinforcement Learning Fundamentals

Reinforcement Learning (RL) is a kind of machine learning where an intelligent agent sequentially chooses actions in an environment. The agent interacts with the underlying environment and learns based on reward signals, which it attempts to maximize over time. Reinforcement learning systems consist of the following main components:

Agent: The learner or decision maker that interacts with an environment. In this research, the agent is the controller that operates the HVAC system.

Environment: The outer system that the agent interacts with. In this project, the environment will be a simulation representing a smart home with an HVAC system. The environment has changing variables that will change conditions, such as indoor temperature, outdoor temperature, occupancy, and time of day.

State: A representation of the current state or condition of the environment perceived by the agent. A state provides the agent with the information needed to make a decision.

Action: A decision or control signal produced by the agent that influences the environment. The agent chooses an action under the belief of the current state of the environment.

Reward: A scalar feedback signal provided by the environment to the agent, received after an action has been executed. The reward indicates the immediate desirability of the agent's action and serves as a learning signal; the agent learns to select actions to maximize the cumulative reward over time.

Reinforcement Learning Algorithms

There are numerous reinforcement learning algorithms, and all of them emphasize the learning of optimal policies in disparate ways. This paper will discuss and compare four of the more well-known algorithms.

Deep Q-Networks (DQN): A value-based RL algorithm employing a deep-learning neural network to approximate the optimal action-value function $Q^*(s,a)$. Since DQNs use Q-learning and are for discrete action spaces, the DQN algorithm includes experience replay and target networks to stabilize learning during training.

Double Deep Q-Networks (DDQN): A variation of DQN. DQNs pick actions to evaluate in a way that consistently leads to overestimation of the Q-values assumed in the selected Q-value. DDQNs explore whether decoupling the selection of the next action from evaluating the Q-value associated with that action provides a benefit. By decoupling the selection of the next action from evaluating its Q-value, DDQNs can reduce expected Q-value

overestimation critically, which supports better policy evaluation.

Proximal Policy Optimization (PPO): An RL policy optimization algorithm that is recognized most for its stability and performance. In the PPO algorithm, instead of having unlimited policy improvement or updates, PPO explains how to update its policy over n time steps with a clipped surrogate objective function that limits a large update to the policy, while allowing small updates. In addition to being conceptually different from DQNs and DDQNs, PPO acts as an actor-critic method, which utilizes a separate neural network for the policy (actor) and value function approximation (critic).

Soft Actor-Critic (SAC) is an off-policy actor-critic algorithm that optimizes a stochastic policy while maximizing entropy. SAC can be used effectively in continuous action spaces, regarded as reliable and sample efficient, attempting to strike a balance between maximizing expected return versus policy entropy to favor exploration.

HVAC Environment: State and Action Definition

In the context of employing reinforcement learning to control HVAC, the environment includes the dynamics of a smart home's thermal behavior and occupancy. The state and action spaces are defined to include all relevant information required for the RL agent to make a control decision.

State: The state of the environment at each time step in regards HVAC describes important parameters that affect thermal comfort and energy consumption. The project defined the state as average temperatures across nine internal temperature sensors (T1-T9), a measure of occupancy based on light usage (where 0 is unoccupied and > 0 is occupied), and hour of day. These components give the agent a snapshot of indoor conditions, occupancy, and time-based context.

Action: The choices of action consist of the discrete control signals that the RL agent can provide to the HVAC system. In this study, actions are defined as operating an HVAC system in one of four modes: Off, Low, Medium, and High. Each mode results in a specific level of energy consumption and heating/cooling output. Energy users can adjust the mode of HVAC operation depending on the state and the learning objectives.

Related Work

Faced with mounting pressure to enhance energy efficiency in buildings, much work has gone into developing improved control strategies for HVAC systems, major energy consumers (Al Sayed et al. 2024; Azuatalam et al.

2020; Lissa et al. 2021). Conventional HVAC control approaches such as ON/OFF control and Proportional Integral Derivative (PID) controllers are not able to proactively accomplish this, and they mostly react to changes in the environment without considering what conflicting objectives (such as energy consumption and occupants' comfort) they need to achieve at one time (Al Sayed et al. 2024). MPC provides a more sophisticated method than traditional control tools because a prediction of future behavior is based on a model of a system's behavior and the control action is optimized over a receding horizon; however, this model requires that the model must be accurate and correctly represent a systems behavior, and optimizing control over a receding horizon is computationally intensive (Al Sayed et al. 2024).

Reinforcement Learning (RL) has gained traction as a promising alternative to traditional control methods, allowing control agents to learn near-optimal policies directly from the environment through interaction, without the restraining need for an explicit system model (Al Sayed et al. 2024; Azuatalam et al. 2020). The fact that RL is model-free is particularly useful in complex and dynamic building settings where control can quickly become overly complicated, too time-consuming, and ultimately impractical to create accurate physics-based models (Azuatalam et al. 2020). Previous work has successfully applied RL to various energy management problems such as whole-building HVAC control in commercial building use cases (Azuatalam et al. 2020), and also for energy management in microgrids with flexible demand (Nakabi and Toivanen 2021) and residential home energy management with heating and domestic hot water and PV self-consumption optimization (Lissa et al. 2021). These types of studies framed their control problem within the context of a Markov Decision Process (MDP), which defined the states as a function of the environmental conditions, occupancy, and time of day; the actions as the control signals to the HVAC or energy systems; and the reward functions as a measure to balance energy efficiency and comfort (Al Sayed et al. 2024; Nakabi and Toivanen 2021; Azuatalam et al. 2020; Lissa et al. 2021).

Numerous RL approaches leveraged through this area have previously been presented (Al Sayed et al. 2024; Nakabi and Toivanen 2021; Azuatalam et al. 2020; Lissa et al. 2021). It is common for value-based algorithms (e.g. Deep Q-Networks and (DQN)), to be used in discrete action spaces, typically using deep neural networks to approximate the action-value function (Al Sayed et al. 2024; Lissa et al. 2021). Actor-critic approaches, which combine policy parameterization and value function estimation, are also common, especially with an eye to continuous action spaces, and many of the actor-critic algorithms, such as Deep Deterministic Policy Gradient, Soft-Actor Critic, and Asynchronous Advantage Actor-Critic, have been applied (Al Sayed et al. 2024; Nakabi and Toivanen 2021). Proximal Policy Optimisation (PPO), an actor-critic algorithm, has

been used for whole-building HVAC control and demand response (Azuatalam et al. 2020). These approaches show significant promise for energy efficiency, reduced costs, and improved thermal comfort compared to conventional approaches (Al Sayed et al. 2024; Azuatalam et al. 2020; Lissa et al. 2021).

This study extends previous work by conducting a thorough comparative study of four different characteristics of some of the most recognized deep reinforcement learning technologies (DQN, DDQN, PPO, SAC) applied to residential HVAC control. Whereas previous studies have either investigated one or subset of these technologies or focused on a different application (Al Sayed et al. 2024; Nakabi and Toivanen 2021; Azuatalam et al. 2020; Lissa et al. 2021), this study allows for a head-to-head comparison between some of these different RL methods applied in a common residential HVAC control problem. Also, it will compare the performance of multiple algorithms when tuning hyperparameters with tools such as Grid Search, providing insights on practical aspects of deploying RL methods in residential HVAC. The research created a dedicated RL environment, tested different algorithms, and compared algorithms for energy savings, cost savings, and temperature deviation, and provided a better understanding of the trade-offs between comfort and energy efficiency operable by different RL technologies in a residential setting.

Preliminaries and Mathematical Framework

Overview of the dataset

This project utilizes the Appliances Energy Prediction dataset from the UCI machine learning repository with 10-minute resolution data spanning 4.5 months. The dataset contains 29 variables, and the most important features are indoor/outdoor temperatures, humidity, and energy consumption. The key features can be broken down into three categories: indoor conditions, outdoor weather, and energy consumption features.

In Figure 1, a Pearson correlation heatmap is shown, including indoor temperatures (T1–T9), computed average indoor temperature (Avg_T1_T9), outdoor temperature (T_out), occupancy status, and derived HVAC energy. It shows that the correlation between indoor temperature sensors and their averages is strongly positive. There is a strong correlation between the derived HVAC energy (Appliances minus lights) and overall energy consumption monitored in the appliances.

In Figure 2, a box plot is shown of HVAC energy consumption per hour of the day (0–23). The plot shows median, 25th and 75th percentiles, interquartile range, and outliers to reveal emerging and settling distinct peaks of energy usage periods in multiple hours, indicating modular

behavioral control and highlighting the RL model's need to balance comfort versus energy consumption.

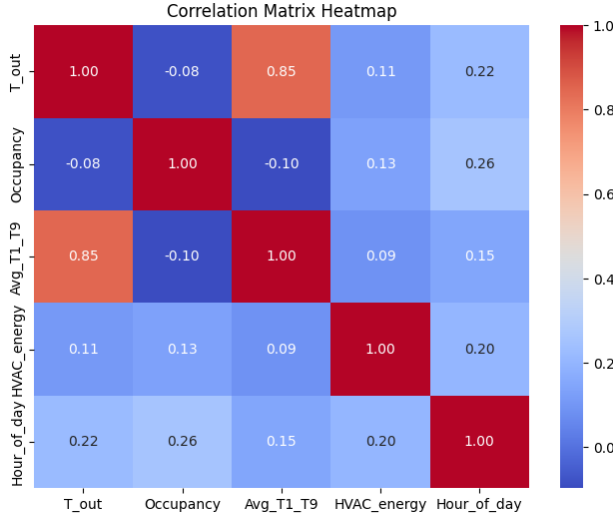


Figure 1: Correlation Matrix

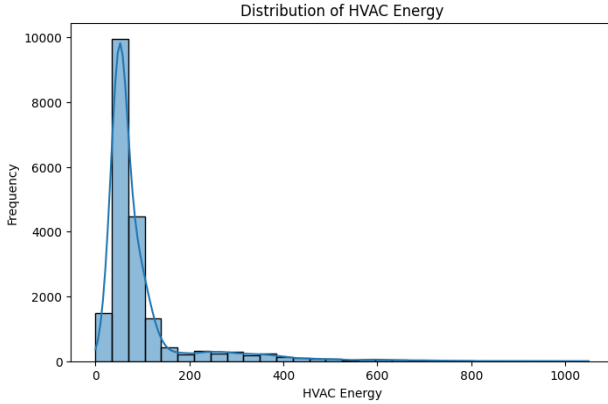


Figure 2: Boxplot of Distribution of HVAC Energy

Defining States and Actions

State space

Each state s is a feature vector that summarizes the current conditions, including the average indoor temperature derived by getting the mean temperature across all 9 rooms, outdoor temperature, occupancy status which is the binary occupancy status derived from lights column (if 0 unoccupied, if lights > 0 occupied), and hour-of-day which is obtained from the time-based context (0-23).

Action Space

The action space is discretized into four distinct HVAC control modes as Off, Low, Medium, and High. To obtain these actions, discrete HVAC modes are mapped to energy thresholds using the percentile method.

Reward function

$$R = -(HVAC_{energy} + \lambda * |Avg(T_1 - T_9) - T_{desired}|) \quad (1)$$

The reward function consists of 3 components: $HVAC_{energy}$ which is the energy consumed by HVAC system (Wh), comfort term, and λ the weighting factor. $HVAC_{energy}$ is derived by subtracting the energy used by lights from the total energy used by the appliances. Comfort term obtained by the difference between the average indoor temperature (T_1-T_9) and the desired indoor temperature. Lambda (λ) is the weighting factor to balance energy vs. comfort (Higher the λ , more comfort, while lower λ , more energy savings).

Key Assumptions

When deriving the states, actions and rewards, a few assumptions have been made.

- HVAC energy is derived by subtracting the energy used by lights from the energy used by appliances. Since the energy used by other household appliances is not included in the data, it is assumed that more than 50% of the household energy is used for heating, ventilation, and air conditioning.
- If the value in the lights column is greater than 0, it is assumed that there are occupants in the room while 0 means there are no occupants in the room. But this has a limitation, if lights are on in unoccupied rooms and lights are off in occupied rooms.
- The temperature in all 9 rooms has been averaged to get the overall indoor temperature.
- It is assumed that the energy ranges for actions (Off, Low, Medium, High) are categorized using percentile methods (0, 25, 50, 75).
- It is assumed that the desired temperature is 22°C. But this could differ in real world situations when the temperature varies through seasons.

This section is the foundation for the reinforcement learning approaches DQN, DDQN, PPO, and SAC, and further optimizes HVAC control for energy savings and improvement of comfort for the occupants.

Methodology

This section outlines the approach executed to solve the problem of optimizing HVAC control with reinforcement learning. It goes over the complete methodology, the separate reinforcement algorithms employed, the architecture of their networks, and the way they are evaluated.

The procedure employs training a reinforcement learning agent to find an optimal policy to control the HVAC system.

The agent works within a simulated HVAC environment by interacting with it, receiving states, taking actions, and potentially gaining rewards. By interacting with the environment, the agent learns to minimize a cost function (negative reward) that incorporates both energy use and temperature deviation. The policy learned can then be deployed into an automated HVAC control function in a real-world application.

The performance of various reinforcement learning algorithms should be explored, Deep Q-Networks (DQN), Double Deep Q-Networks (DDQN), Proximal Policy Optimization (PPO), and Soft Actor-Critic (SAC), which represent different types of RL methodologies which including value-based (DQN, DDQN) and policy optimization (PPO, SAC).

Deep Q-Networks (DQN) and Double Deep Q-Networks (DDQN)

DQN and DDQN both operate with Q-Networks that estimate the action-value function $Q(s,a)$. The network takes the state as input and applies its weights to come up with values representing Q-values for each action. The Replay Buffer conforms, stores tuples of an experience of the form $(s_t, a_t, r_t, s_{t+1}, done_t)$ and samples mini-batches that can be used for training. The replay buffer breaks correlations in the data, which helps to increase stability in learning.

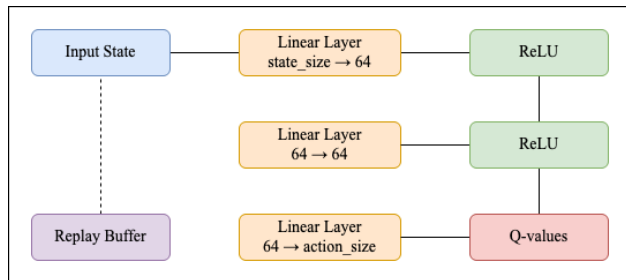


Figure 3: DQN and DDQN Architecture

The Q-Network defines three fully connected (Linear) layers with two ReLU operations between the first two layers, a layer of output is equal to the size of the number of possible actions. The experience is stored by the deque of a specific capacity to remember all transitions.

- Layer 1: Linear (state_size, 64)
- Activation: ReLU
- Layer 2: Linear (64, 64)
- Activation: ReLU
- Layer 3: Linear (64, action_size)

The deque contains useful methods to push new experiences and sample random mini-batches. Both DQN and DDQN use the Mean Squared Error (MSE) with respect to the predicted and target Q-values. The target Q-value

calculation is a little different for DQN and DDQN to alleviate the overestimation bias found for estimating Q-values in DQN.

Proximal Policy Optimization (PPO)

PPO is an actor-critic algorithm that maintains a separate network for the actor (policy) and the critic (value function). The actor outputs the probability distribution over actions - i.e., it defines the probability of taking each action - while the critic provides an estimate of the value of the current state.

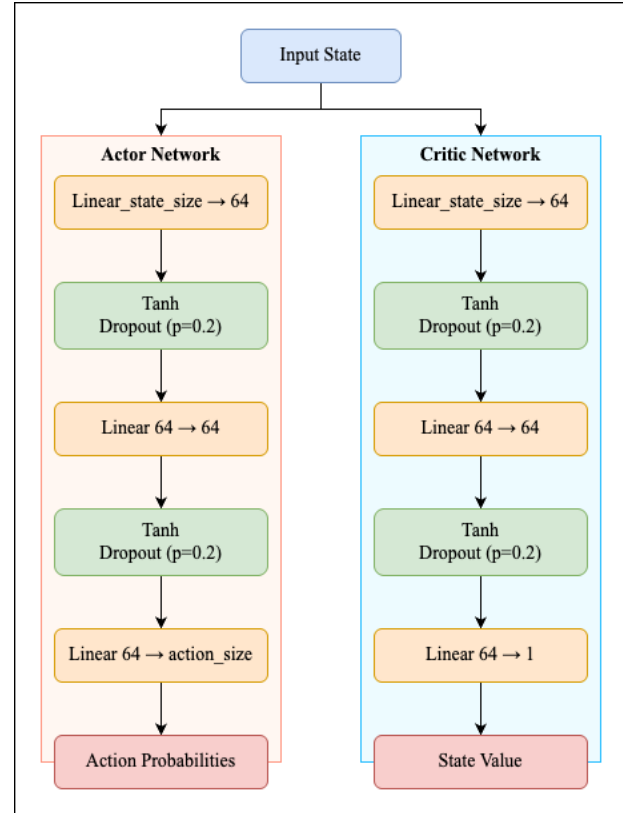


Figure 4: PPO Architecture

The Actor network receives as input a 1D array that represents the observation (state) provided by the environment. The Actor network consists of three fully connected (Linear) layers that utilize Tanh activation functions. There is also a dropout with probability 0.2 after the first layer and the second layer. The output layer has a size equal to the number of possible actions, which results in logits corresponding to a categorical distribution over the actions.

- Layer 1: Linear (state_size, 64)
- Activation: Tanh
- Dropout (p=0.2)
- Layer 2: Linear (64, 64)

- Activation: Tanh
- Dropout (p=0.2)
- Layer 3: Linear (64, action_size)

The Critic network receives the identical observation (state) used for the Actor network and has a similar structure. The Critic network is also three fully connected (Linear) layers that have Tanh activation functions, with dropout (probability 0.2) after the first layer and the second layer. The output layer is a single neuron and provides an estimate of the value for the input state.

- Layer 1: Linear (state_size, 64)
- Activation: Tanh
- Dropout (p=0.2)
- Layer 2: Linear (64, 64)
- Activation: Tanh
- Dropout (p=0.2)
- Layer 3: Linear (64, 1)

PPO minimizes a clipped surrogate objective function that encourages updates to the policy while also preventing large updates (in order to stabilize the update). The critic is usually trained via MSE to minimize the difference between the predicted value to the target value (e.g. General Advantage Estimation).

Soft Actor-Critic (SAC)

SAC, or Soft Actor-Critic, is an off-policy actor-critic algorithm that seeks to maximize an expected return and an entropy trade-off. SAC consists of an actor, two Q-networks (which help reduce overestimation), and a value function. In addition, there is a Replay Buffer.

The Actor consists of three fully connected (Linear) layers with ReLU activation functions. The Actor output layer consists of logits for a categorical distribution over actions.

- Layer 1: Linear (state_size, 64)
- Activation: ReLU
- Layer 2: Linear (64, 64)
- Activation: ReLU
- Layer 3: Linear (64, action_size)

The two Q-networks (Q1, Q2) share the same architecture as the Actor. Each consists of three fully connected (Linear) layers with ReLU activation functions. The output layer consists of an output size of the possible actions.

- Layer 1: Linear (state_size, 64)
- Activation: ReLU
- Layer 2: Linear (64, 64)
- Activation: ReLU
- Layer 3: Linear (64, action_size)

The Value function estimates the value of the current state and consists of three fully connected (Linear) layers with ReLU activation functions. The output layer consists of a single neuron.

- Layer 1: Linear (state_size, 64)
- Activation: ReLU
- Layer 2: Linear (64, 64)
- Activation: ReLU
- Layer 3: Linear (64, 1)

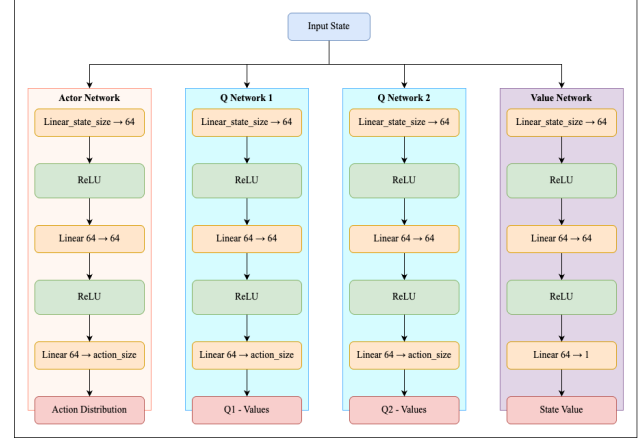


Figure 5: SAC Architecture

Similar to DQN and DDQN, the replay mechanism is implemented using a deque. SAC has many loss functions with the actor, Q-networks, and value networks. The actor aims to maximize the expected value while maximizing entropy. The Q-networks are trained with a variation of the Bellman equation, where the value network is trained to predict expected return.

To evaluate the performance of the trained RL agents, the following metrics will be used:

Total Energy Consumption: The total amount of HVAC energy used over a fixed interval (e.g., a day or week). The lower the number, the better the energy use.

$$E_{total} = \sum_{t=1}^T HVAC_{energy,t} \quad (2)$$

Where T indicates the total time steps in the evaluation interval.

Total Cost: The total monetary cost to the user associated with HVAC energy use. The total cost can be calculated by multiplying the total energy use by the price of energy that may change depending on the time of day.

$$C_{total} = \sum_{t=1}^T HVAC_{energy,t} \times Price_t \quad (3)$$

Where $Price_t$ is the price of energy at time step t .

Temperature Deviation: The average absolute difference between the actual indoor temperature and desired temperature over a defined interval. The lower the metric, the better temperature regulation and comfort.

$$TempDev_{avg} = \frac{1}{T} \sum_{t=1}^T |AvgT_{1-9,t} - T_{desired}| \quad (4)$$

The comfort index is a composite metric that accounts for both temperature deviation and discomfort factors (i.e., large temperature fluctuations). The higher the comfort index the more satisfied are the occupants. The specific form of the comfort index can vary or be adjusted based on the needs of the project.

These variables will be used to compare how well different RL algorithms perform and how well the algorithms are achieving the goals of the project.

Experiments

This section describes the experimental design, plan of action, and results obtained by applying several reinforcement learning algorithms to the HVAC control problem. The primary goal of the experiments was to compare the performance of various algorithms and if they were able to achieve energy and cost saving with acceptable indoor temperature levels, after optimizing algorithm hyperparameters.

The experiments were completed using a cloud computing instance with the required computational resources. A cloud instance was provisioned with 4 CPU cores, 28GB of RAM, and 56GB of disk storage. The Azure Machine Learning platform was used with the VS Code Extension, which offered a robust model for developing, training, and managing the machine learning models. Terminal multiplexers, like tmux or screen were used to manage the experimental process workflow. The implementation was exclusively completed in the Python programming language. Data preprocessing involved dataset cleaning and transformations were completed using Pandas and NumPy, while DQN, DDQN, PPO, and SAC deep reinforcement learning model implementations were started using the PyTorch library. Data splitting and KFold cross validation for hyperparameter tuning were completed via scikit-learn functions. Experiment results, including performance metrics and configuration file information, were saved and managed as JSON formatted files. Data visualizations were produced using Matplotlib.

The experimental plan also incorporated a thorough evaluation of the targeted reinforcement learning algorithms. For analysis, the primary dataset used a time series split, where the first 70% of the data was used to train the RL agents ("known data"), and the second 30% of the data was used as unseen data ("test data") for the evaluation. For tuning the experimentation with the RL algorithms,

several methods would be used, like Grid Search and possibly KFold cross-validation.

Each RL algorithm would be set up with different training configurations and hyperparameters. For example, the DQN algorithm would use a discount (Gamma) factor of 0.95, an initial epsilon-greedy method exploration state of 1.0, with a decaying factor of 0.995 at each step, down to a minimum of 0.01; batches of training 32, a learning rate of 1e-05, and frequency of training would be every 4 runs, and the DDQN parameterization were all the same, but differed in the Gamma factor of 0.90, an initial epsilon exploration state of 1.0, with a decaying factor of 0.995, down to a minimum value of 0.01, batch of training 32, a factor of learning rate of 1e-05, and frequency of training every 4 runs. DDQN also included a replay buffer of size 5000.

Model	Hyperparameters
DQN	Gamma: 0.95 Epsilon: 1.0 Epsilon decay: 0.995 Epsilon min: 0.01 Batch size: 32 Learning rate: 1e-05 Training frequency: 4
DDQN	Gamma: 0.9 Epsilon: 1.0 Epsilon decay: 0.995 Epsilon min: 0.01 Batch size: 32 Buffer size: 5000 Learning rate: 1e-05 Training frequency: 4
PPO	Clip Epsilon: 0.2 Discount Factor (γ): 0.95 GAE Lambda (λ): 0.95 Actor Learning Rate: 1e-4 Critic Learning Rate: 1e-3 Number of Update Epochs: 4 Mini Batch Size: 64 Entropy Coefficient: 0.01 Dropout Rate in Networks: 0.2 Weight Decay: 1e-4
SAC	Gamma: 0.93 Alpha: 0.38 Learning rate: 0.00078 Batch Size: 64

Table 1: Model Hyperparameters

The PPO algorithm was set with a Clip Epsilon of 0.2, a Discount Factor (γ) of 0.95, and a GAE Lambda (λ) of 0.95. The Actor network was trained using a learning rate of 1e-4, while the Critic network was trained using a learning rate of 1e-3. Training was performed using 4 update epochs for

each policy rollout, using mini batches of size 64. An Entropy Coefficient of 0.01 was used, with a Dropout Rate of 0.2 applied in the networks, and the Weight Decay was set to $1e-4$. For the SAC algorithm, a Gamma of 0.93 and an Alpha of 0.38 were set. For training the SAC algorithm, a learning rate of 0.00078 was used and a Batch Size of 64 was used.

The trained RL agents' performances were evaluated in multiple dimensions. Average Rewards, Energy Savings, Temperature Deviation, and Cost Savings. The experiments included evaluation of both the standard implementations of each algorithm, with corresponding configurations obtained from hyperparameter tuning.

The setup and framework for the experiment provided a defined way to evaluate the performance of the candidate reinforcement learning algorithms in an organized manner, as well as to assess the multiple specifications for how to accomplish energy savings and comfort provided throughout the project.

Results

This section contains the outcomes from the experiments carried out with several reinforcement learning algorithms trained and evaluated on HVAC control tasks. Each algorithm was evaluated using several relevant metrics to characterize its performance. The analysis around the comparative performance of the algorithms allowed us to identify the best strategy for effectively controlling HVAC energy use while still achieving an acceptable occupant comfort level. Beyond relying on metrics to evaluate the differences in performance, we also looked at how the hyperparameters were optimized for every algorithm and how that has impacted performance as well.

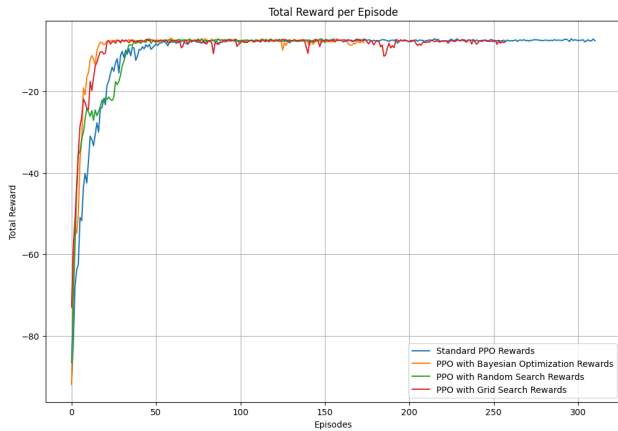


Figure 6: PPO Total Rewards per Episode

In addition, Table 2 summarizes the results for the different reinforcement learning algorithms tested in the

different experiments, along with their standard implementations and hyperparameter-tuned variants. The results in the table are reported based on Average Total Reward, Average Energy Savings (Wh), Average Temperature Deviation, and Average Cost Savings.

When compared against each other, the differing reinforcement learning algorithms each had different strengths and weaknesses when controlling HVAC systems. Across representations, the PPO algorithm generally achieved the highest energy and cost savings. Standard PPO (Exp 1) and PPO using Grid Search (Exp 4) achieved the best energy savings and exceeded savings of over 41,000 Wh. With regard to temperature, the alternatives (PPO variants) had generally larger temperature deviations than the alternative algorithms.

The DQN and DDQN algorithms were competitive on energy and cost savings, though generally lower than the best PPO outcomes. Standard DQN (Exp 5) had the lowest average reward of all the DQN and DDQN variants. Optimized DQN (Exp 6) and Optimized DQN (Exp 8) did better than standard DQN and DDQN for most metrics, suggesting that tuning yields a benefit. DDQN, particularly its optimized version (Exp 8), achieved a good tradeoff between energy savings and temperature deviation, having a reasonable temperature deviation of 2.86 degrees.

The SAC algorithm had much lower energy and cost savings than PPO, DQN, and DDQN, but had the lowest average temperature deviations. Standard SAC (Exp 9) had a temperature deviation of 1.05, whereas Optimized SAC (Exp 10) had a higher temperature deviation that was lower (0.56). Therefore, SAC potentially sacrificed energy consumption for temperature regulation and better maintained the desired temperature than the other algorithms, as indicated by its lower average rewards.

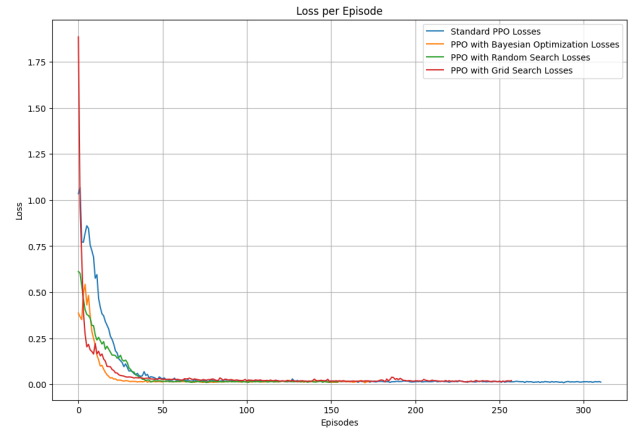


Figure 7: PPO Loss per Episode

The experimental results clearly demonstrate that hyperparameter optimization can strongly influence the performance of reinforcement learning algorithms for HVAC control. The optimized versions of DQN and DDQN

always had higher average rewards, energy/cost savings, and lower temperature deviations than their standard versions. Therefore, a fine-tuned set of hyperparameters makes a big difference to performance.

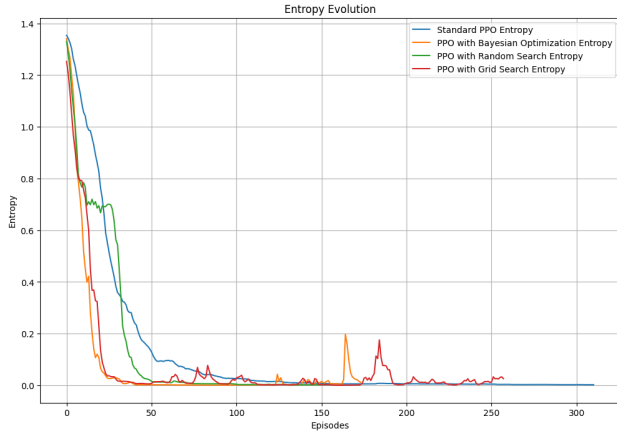


Figure 8: PPO Entropy Evolution

The results suggest that different hyperparameter tuning methods (Bayesian Optimization, Random Search, Grid Search) had different effects on the performance within each of the PPO experiments. Standard PPO and PPO with Grid Search, for example, had the highest energy savings among all the PPO variations, while the differences in average rewards and temperature deviation among the strains of PPO were not as notable in comparison to the differences seen between optimized and standard DQN (Exp 5 vs Exp 6) and DDQN (Exp 7 vs Exp 8). This could imply that the range of hyperparameter tuning methods used within this project does not greatly impact PPO's performance or that the standard hyperparameters were already somewhat well-tuned for the specific environment.

The most significant effects of hyperparameter tuning were observed for the SAC model. Optimized SAC (Exp. 10) achieved a significant increase in average reward as well as the lowest temperature deviation out of all the experiments while also increasing energy savings as well a cost savings compared to Standard SAC (Exp. 9). This would suggest that SAC is more sensitive to its hyperparameters and consequently tuning the hyperparameters is important in order to achieve the desired performance.

Generally, results support a trade-off between energy efficiency and temperature regulation depending on the algorithm and its settings. The PPO variants consistently did the best job of maximizing energy and cost reductions, while the SAC variant fitted a control policy that was as close as possible to a temperature setpoint, so it minimized the temperature deviation from the setpoint. The optimized algorithms consistently outperformed the standard versions, indicative of the importance of hyperparameter tuning. Ultimately, the decision of what algorithm is "best" depends on specific priorities, - minimize energy consumption, or minimize deviation from temperature setpoints.

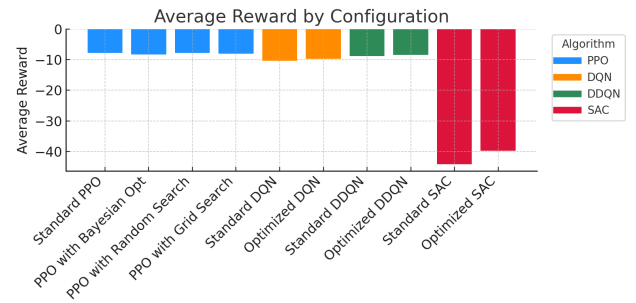


Figure 9: Average Reward across experiments

Experiment	Description	Average Rewards	Energy Savings	Temperature Deviation	Cost Savings
Exp 1	Standard PPO	-7.86	41483.68	3.53	4148.37
Exp 2	PPO with Bayesian Optimization	-8.37	40934.45	3.53	4147.68
Exp 3	PPO with Random Search	-7.85	41099.92	3.48	4109.99
Exp 4	PPO with Grid Search	-8.11	41497.53	3.54	4149.75
Exp 5	Standard DQN	-10.36	38977.20	3.17	3897.72
Exp 6	Optimized DQN	-9.80	40250.50	3.00	4025.05
Exp 7	Standard DDQN	-8.89	40558.17	3.39	4055.82
Exp 8	Optimized DDQN	-8.50	41200.00	2.86	4120.00
Exp 9	Standard SAC	-44.10	4171.77	1.05	417.18
Exp 10	Optimized SAC	-39.76	4678.56	0.56	467.85

Table 2: Experiment Results

Discussion

This section will provide an interpretation of the experimental outcomes, discuss some of the difficulties faced during the research, and then highlight the limitations of the current study.

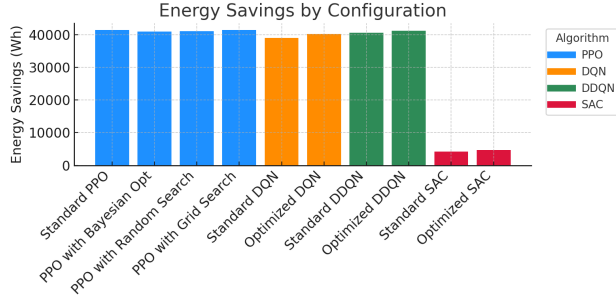


Figure 10: Energy savings across experiments

The experimental results demonstrate that reinforcement learning has great potential for optimizing HVAC control for the purpose of lowering energy use and cost when some allowance can be made for temperature deviations. The results of the experiments show that PPO with Grid Search had the best performance, in either controlling for some temperature deviations or it had the best performance in terms of energy/cost savings. Experiment 4 represents PPO with Grid Search, which produced energy savings of 41497.53 Wh and cost savings of 4149.75. Both of these two amounts represented the highest savings out of all of the experiments that were run. The average temperature deviation was 3.54°C, which is a little higher than other models such as Optimized SAC (0.56°C) or Optimized DDQN (2.86°C), but when energy and cost savings are foremost in mind, along with not having excessive discomfort because of temperature, the deviations become very acceptable. This indicates that when tuned effectively, as shown here with the Grid Search, PPO is able to learn a policy that equally minimizes energy use. The average reward of -8.11 for PPO with Grid Search indicates good academic performance when accounting for energy use and temperature penalty components of the reward function. Again, the improvement of the optimized versions of DQN, DDQN, and SAC over standard versions further emphasizes the importance of tuning hyperparameters to mitigate the complexity associated with an HVAC control problem, especially when there is noise.

There were a few challenges throughout the research process. The first was with the compatibility between Gym and Gymnasium (particularly when using Stable Baselines3 [SB3], which moved to Gymnasium) and some shimmies import errors with the classic OpenAI Gym environments. This was resolved by controlling library versions, and environments. A second challenge and an important challenge was on time and computational expenses on hyperparameter searches because the grid, random, and

Bayesian optimization were all computationally more expensive and increased training times.

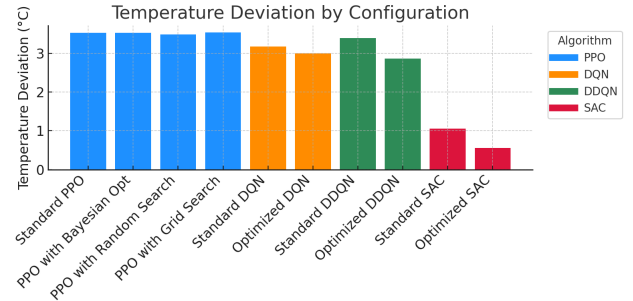


Figure 11: Temperature deviation across experiments

Managing algorithm stability and consistent convergence across the various RL algorithms (DQN, DDQN, PPO, and SAC) during the noisy HVAC simulation environment was not an easy task. Techniques like careful regularization of the network through dropout layers and weight decay, in an effort to control overfitting and push the network to learn stronger features, were used. We used early stopping to weight the validation performance and stop the training when applicable measures plateaued to also assist in reducing overfitting. Limits to computer resources for the training, such as a 4-core, 28 GB RAM Azure instance, can create resource bottlenecks and influence turnaround for experiments.

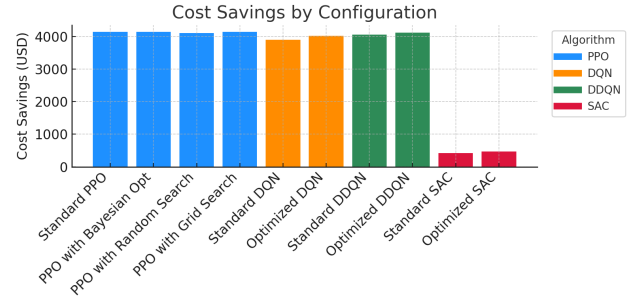


Figure 12: Cost savings across experiments

This work has its limitations. The most notable limitation is the limitations of the data. The dataset only had features available, which would inevitably lead to assumptions about how the dataset would operate, which could affect the accuracy and truthfulness of the model in a real-life scenario. At times we conducted the experiments in a simulated HVAC environment, essentially extending the model from the dataset; the exploration, deployment, and evaluation were not achieved in a real-world smart home environment. Therefore, the performance that we observe in simulation may not accurately transfer due to unforeseen variables in complexity and variability during the real-world testing.

Moreover, although hyperparameter tuning took place, the search space may not have been fully explored, and with further refinements and/or hybrid optimization exploration,

the outcomes may have been superior. Long-term performance metrics can also be incorporated, in addition to the development of a wider comfort index dealing with variables not described simply in terms of temperature deviation. Also, the absence of an easy-to-use application for seamless integration into existing smart home systems hampers the immediate practical application of the RL-based control strategy developed.

Future Work

Given the results and shortcomings of this study, there are many potential avenues that could be explored for future research and development, as these pathways will seek to advance the present work in order to maximize the impact and real-world applicability of reinforcement learning for HVAC control.

One possible path for future work is further polishing of hyperparameter tuning methodologies. The possibility for a significantly more effective approach for controlling HVAC systems is a few more refined hyperparameter tuning strategies away. Considering other more sophisticated, or hybrid methods of optimization beyond Grid Search, Random Search, and Bayesian Optimization might yield superior hyperparameter configurations and ultimately improve the trade-off between energy use and occupant comfort.

Another important avenue is developing a user-friendly application that integrates into existing smart-home applications. An application that allows the RL agent to monitor and control the HVAC system in real-time, and also provide users with visualizations of energy use, monetary savings, occupant comfort metrics, could drive the real-world application of this RL-based control, allowing users to better understand the RL agent's decisions and better trust its decisions.

Moreover, broadening the scope of evaluation towards long-term field studies is important. The present study provides value based on historical data and simulated research; however, deploying and testing the trained agents in real homes over a longer period would be a more comprehensive assessment of their robustness, ability to adapt to dynamic unhidden conditions, and continuing ability to achieve energy savings and comfort objectives. It might also be valuable to identify challenges that are not evident in simulation.

Conclusion

In summary, this study investigated the use of reinforcement learning to optimize HVAC control for smart homes with the goal of minimizing energy consumption, the associated cost of energy consumption while maintaining stable indoor temperatures. The study framed the HVAC control problem as a reinforcement learning problem, which

included defining the state space, action space, and a reward function that would simultaneously consider energy efficiency and comfort. An experiment was designed to evaluate four algorithms: Deep Q-Network (DQN), Double Deep Q-Network (DDQN), Proximal Policy Optimization (PPO), and Soft Actor-Critic (SAC). The analysis of experimental results indicated that the PPO algorithm optimized using Grid Search was a very strong algorithm for the purpose of this study, which saved a large amount of energy usage and costs while having acceptable temperature deviations. The results provided an indication of reinforcement learning's capability to learn intelligent control policies that can be adapted to changing environmental conditions to optimize HVAC operation, which is important in reducing energy waste and negative environmental outcomes. Although challenges were noted with respect to algorithm stability and computational resources as well as limitations related to data, the successful optimization and results of these RL models, including, in particular, the PPO model, are considerable and provide a strong basis for future work, including real-life implementation and further developing control strategies.

References

- Al Sayed, K. A.; Boodi, A.; Sadeghian Broujeny, R.; and Beddiar, K. 2024. Reinforcement learning for HVAC control in intelligent buildings: A technical and conceptual review. *Journal of Building Engineering*, 95: 110085. DOI: 10.1016/j.jobe.2024.110085.
- Azuatalam, D.; Lee, W.; De Nijs, F.; and Liebman, A. 2020. Reinforcement learning for whole-building HVAC control and demand response. *Energy and AI*, 2: 100020. DOI: 10.1016/j.egyai.2020.100020.
- Lissa, P.; Deane, C.; Schukat, M.; Seri, F.; Keane, M.; and Barrett, E. 2021. Deep reinforcement learning for home energy management system control. *Energy and AI*, 3: 100043. DOI: 10.1016/j.egyai.2020.100043.
- Nakabi, Taha Abdelhalim; and Toivanen, Pekka. 2021. Deep reinforcement learning for energy management in a microgrid with flexible demand. *Sustainable Energy, Grids and Networks*, 25: 100413. DOI: 10.1016/j.segan.2020.100413.